- Updated agent color palette
- Fixed `/goal` silently hanging when `disableAllHooks` or `allowManagedHooksOnly` is set — now shows a clear message instead of an indicator that never resolves
- Fixed a regression in settings hot-reload where symlinked settings files caused misattributed change events and spurious `ConfigChange` hooks
- Fixed `claude --bg` failing with "connection dropped mid-request" when the background service was about to idle-exit
- Fixed background service startup failing on machines with enterprise endpoint security by allowing more time
- Fixed remote managed settings not retrying on 401 — now retries once with a force-refreshed token
- Fixed managed `extraKnownMarketplaces` auto-update policy not being persisted to `known_marketplaces.json`
- Fixed `/loop` scheduling redundant wakeups to poll for background tasks that already notify on completion
- Fixed a recurring event-loop stall on Windows when a missing executable (e.g. `gh`) triggered synchronous `where.exe` re-spawns on every check
- Fixed `Read` tool calls failing validation when `offset` is passed as a whitespace-padded or `+`-prefixed string
- Fixed native terminal cursor not staying at the input caret when the terminal loses focus
- Plugins now warn when a default component folder (e.g. `commands/`) is silently ignored because `plugin.json` sets the matching key. Shown in `/doctor`, `claude plugin list`, and `/plugin`.

FRED TALKS

13th May 2026

CONTENTS

The Self-Help Trap: What 20+ Years of “Optimizing” Has Taught Me

TIM FERRISS · 08 MAR 2026 · SOURCE

One danger of modern self-help.

> **“We cannot reason ourselves out of our basic irrationality. All we can do is to learn the art of being irrational in a reasonable way.”**
> — Aldous Huxley, *Island*

It was cold out, but none of us were cold.

I sat with five men in the mountains of Montana. As the sun set, the fire in the center cast dancing light on our faces. Reclined against fallen trees in a tight circle, we ate mushrooms and fish we’d found under trees and along streams. The whole crew burst into laughter yet again, and one of the guides passed around a fresh batch of pine needle tea.

Bathed in warmth, I took off a layer and glanced skyward through an opening in the trees. The stars shone like crystals on black velvet, and the show—the biggest meteor shower of the year—was starting.

In that moment, there was nothing to do. Nothing to improve. Nothing to fix.

It was perfect.

***

The older I get, the more I think that self-help can be a trap. Sometimes the cure is worse than the disease. I say this after ~20 years of writing self-help and a lifetime of consuming it.

Spend enough time in the world of “improvement,” and you’ll notice something strange: The people most obsessed with self-help are often the least helped by it. Behind the smiles and motivational quotes, behind closed doors and after a drink or two, the truth is that they’re not able to outsmart their worries.

On one hand, perhaps this unhappiness is precisely what lands one in self-development in the first place, right? I long assumed this about myself, and it’s partially true.

On the other hand, what if self-help *itself* is actually creating or amplifying unhappiness?

You had anticipated this and not given it access to the FooBarDB client library; and marked the FooBarDB CLI as non-executable; but it just went ahead and used curl to access a forgotten REST endpoint.

The Rogue Agent: As the agent copies state (this time using the CLI tool after you yell at it enough), it lists keys in batches; each key corresponds to some user of your application. Someone decided to pick the user name “delete-everything”. Of course, you are using the latest model which is always resistant to prompt injections. Almost always.

The Stupid Agent: You do everything right. There are no prompt injections in your state. You wall off FooBarDB and force the agent to use the CLI tool. The model decides to delight you by generating an unusual and interesting sequence of tokens that will optimally save storage space for you, translating to a lambda that deletes your production data entirely.

One response to these difficult failure modes might be: wait until the next model. Maybe a smarter model will never delete all of your production data, or implement a slow locking protocol. But even the smartest model is subject to the four horsemen of the distributed apocalypse: asynchrony, crashes, concurrency, and network partitions.

In recent work called LogAct, my colleagues and I take a first step towards solving this problem. We go back to the difference between vibe coding and vibe engineering: git is *transactional*, infrastructure is not. How do we make infra more transactional, like git? We propose that an agent should be a ***deconstructed state machine on a shared log***, borrowing ideas from distributed systems (State Machine Replication, Byzantine Fault-Tolerance) and databases (Atomic Commit, Write-Ahead Logging). LogAct can stop unsafe actions before they happen (by collecting votes on the shared log); recover from crashed actions (using the shared log as a WAL); and provide an audit trail for actions after they complete.

But this is just one step. A self-writing program is a strange, astonishing, and novel creature; making it reliable will require innovation in every branch of Computer Science!

2.1.140

CHANGELOG · 12 MAY 2026 · SOURCE

- Improved Agent tool `subagent_type` matching to accept case- and separator-insensitive values (e.g. `"Code Reviewer"` resolves to `code-reviewer`)

So, a re-restatement of Agentic Fault-Tolerance: *How do we make a self-writing program fault-tolerant when it acts upon real-world environments?*

To answer this question, we examine the many types of failures that agents can experience.

The Crashed Agent: You tell an agent to move a bunch of data from FooBarDB to S3 for freeing up space. It generates a lambda to move this data and chugs along moving files. Midway through, the agent crashes. How do you recover?

The Zombie Agent: You start a new agent on a different server. Happily, the previous agent crashed after the copy but before the delete; so a new agent is able to re-execute the command in an idempotent manner. Sadly, the files are now very slow to access; you change your mind and tell the agent to move them back to FooBarDB, which it does successfully. You go get a cup of coffee. Unfortunately, it turns out that the previous agent had not actually crashed; it was just temporarily partitioned away on the network. It wakes up and continues executing its logic, deleting the (now only) copy of state from FooBarDB.

The Dining Agent Philosophers: While you were telling your agent to copy state from FooBarDB to S3, someone else on the team had the same idea and started another agent to do the exact same thing. Now you have two copies of state in S3. Of course, you could have solved this problem by appointing the one and only Agentic Oncall Czar responsible for running the single agent that can touch production; but then if that agent crashes, now you have a potential Zombie Agent.

The Slow Agent: You tell your agent to follow a sophisticated locking protocol before accessing the FooBarDB table. This works to avoid concurrency bugs… but the agent decides to lock each file individually by writing to a conditional register, which takes forever. At some point you kill the agent, turning a Slow Agent into a Crashed Agent.

The Lazy Agent: You tell the agent to try locking again, except this time monitor its own performance over batches of files and pick efficient locking protocols. The agent determines that the best way to optimize locking is to just not do it, because it seems quite unlikely that someone else would be accessing these files at the same time.

The Clever Agent: Of course, everyone on your team is a distributed systems expert; and so someone built a CLI tool in 2018 for safely transferring your application's state, with support for locking and write-ahead logging in case of failures; there's even a sentence in your onboarding doc saying "NEVER TOUCH STATE IN FOOBARDB DIRECTLY!". You even told the agent explicitly about this tool. But time passed, context filled up, the agent forgot about this tool's existence, or maybe it just preferred FooBarDB's in-distribution API to your esoteric CLI tool.

Modern self-help contains an in-built flaw:

To continually improve yourself, you must continually locate the ways you are broken.

Fortunately, there are a few perspective shifts that make all the difference. It took me embarrassingly long to figure them out.

To get started, let's take a fresh look at an old concept.

MASLOW'S HIERARCHY OF NEEDS?

> **"I suppose it is tempting, if the only tool you have is a hammer, to treat everything as if it were a nail."**
> — Abraham Maslow

Maslow's Hierarchy of Needs has captured the minds of hundreds of millions. It offers simplicity in a terrifyingly complex world.

Abraham Maslow's "A Theory of Human Motivation" (1943) contains five levels, which are typically presented like the below pyramid. This one is pulled from the Wikipedia entry on the subject:

We've all seen it. Clear as day, you can see the goal post at the top: self-actualization.

LFG! It's time to journal and 80/20 myself! Pass me a shaman and some modafinil.

That's the mission. That's the point.

Right?

But hold on. A critical footnote got lost in the shuffle. In his later writings, especially notes compiled in *The Farther Reaches of Human Nature* (1971), Maslow added a *sixth* level above self-actualization:

Self-transcendence.

That update never quite made it out of the crib. The consultants are to blame, but that comes later.

Self-transcendence means going beyond the self — seeking connection with something greater, such as service to others, nature, art, or the divine. Why is it important? Well, for one thing, as Tony Robbins put it at an event long ago: "'I, I, I, me, me, me' gets to be a really fucking boring song."

But it's not just a boring song; it's dangerous to your health.

DON'T BE A SOMO

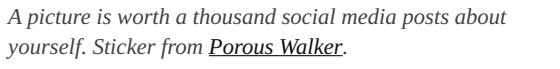
> **"The man who renounces himself, comes to himself."**
> — Ralph Waldo Emerson

Self-help is dangerous precisely because it easily becomes self-fixation.

A focus on improving the self usually first requires finding problems with the self. This is quite the pickle. In a society that rewards problem-solving, you can end up hallucinating or exaggerating unease in order to fix it. This leaves you always in the red, always one step behind. Imagine a dog chasing its tail that has committed to being unhappy until it catches the tail… but it's always just a few inches short. Still, it whirls around and around, "doing the work." Perfection always recedes by one more book, one more seminar, one more habit tracker.

Put in more colorful terms, misdirected self-help turns you into a self-obsessed masturbatory ouroboros (SOMO).

To remind me of the SOMO risk, I have this sticker on my laptop:



A picture is worth a thousand social media posts about yourself. Sticker from Porous Walker.

Now, to be clear, I still love self-help. Ain't no way Timmy can give up the sauce. There's a place for it.

From The Bible to Seneca, and from Ben Franklin to Stephen Covey and far beyond, there's a lot of valuable advice worth taking. I used to mainline it all – no time to waste! – and jump straight into action. This did some good, but there was a lot of collateral damage.

Why?

Because there are at least three "tectonic plates of self-help" that I couldn't see for decades, and they dictate how much net-positive or net-negative comes from all the striving. Before you sprint, you want to calibrate your direction.

Postscript: This blog seems to have resurfaced 7 years later. I actually still agree with much of it, though the dilemma is much less important. Distributed systems operations is hard, but you can now avoid it by just getting your distributed systems as a service. For example, Confluent offers Kafka as a service with a contractural uptime SLA. Ops remains hard, but for distributed data systems you can make the people who wrote the software deal with the problem, and focus your time on your special sauce.

Your Agent is a Distributed System (and fails like one)

MAHESH'S BLOG · 24 APR 2026 · SOURCE

We start with a re-definition and an observation.

A common definition of an agent is that it's an LLM that calls tools in a loop. This definition is incorrect. An agent is not an LLM. It is not restricted to calling tools. It does not particularly have to be a loop.

Instead, a better definition of an agent is that it's a self-writing program. If a program is defined as a sequence of states, moving from one state to another by executing a lambda, then an agent is a program that uses an external LLM to determine the next lambda to execute / the next state in this sequence.

Accordingly, the new problem of Agentic Fault-Tolerance can be restated as: *How do we make a self-writing program fault-tolerant?*

Now, the observation: **vibe coding != vibe engineering**. In vibe coding, the agent / self-writing program acts primarily upon a code repository. Code repositories are forgiving: an agent can hallucinate, create unnecessary files, delete critical state, remove tests entirely, fail in the middle, forget what it was doing, reward-hack in bizarre ways, completely trash a code repo; all you have to do is git reset.

In vibe engineering, the agent acts upon infrastructure (S3 buckets, DynamoDB tables, EC2 instances, K8s deployments…). Large-scale infrastructure is not forgiving. There is no reset. Only SEVs at 2 AM.

I would love to see claims in academic publication around practicality or reliability justified in the same way we justify performance claims–by doing it. I would be a lot more likely to believe an academic distributed system was practically feasible if it was run continuously under load for a year successfully and if information was reported on failures and outages. Maybe that isn't feasible for an academic project, but few other allegedly scientific academic disciplines can get away with making claims about reality without evidence.

More broadly I would like to see more systems whose *operation* is verifiable. Many systems have the ability to log out information about their state in a way that makes programmatic checking of various invariants and guarantees possible (such as consistency or availability). An example of such an invariant for a messaging system is that all the messages sent to it are received by all the subscribers. We actually measure the truth of this statement in real-time in production for our Kafka data pipeline for all 450 topics and all their subscribers. The number of corner-cases one uncovers with this kind of check, run through a few hundred use cases and a few billion messages per day is truly astounding. I think this is a special case of a broad class of verification that can be done on the running system that goes far far deeper than what is traditionally considered either monitoring or testing. Call it unit testing in production, or self-proving systems, or next generation monitoring, or whatever, but I think this kind of deep verification is something that makes turning the theoretical claims a design makes into measured properties of the real running system.

Likewise if you have a "NoSQL vendor" I think it is reasonable to ask them to provide hard information on customer outages. They don't need to tell you who the customer is, but they should let you know the observed real-life distribution of MTTF and MTTR they are able to achieve, not just highlight one or two happy cases. Make sure you understand how they measure this, do they have automated test load that runs or just wait for people to complain? This is a reasonable thing for people paying for a service to ask for. To a certain extent if they provide this kind of empirical data it isn't clear why you should even *care* what their architecture is beyond intellectual curiosity.

Distributed systems solve a number of key problems at the heart of scaling large websites. I absolutely think this is going to become the default approach to handling state in internet application development. But no one benefits from the kind of irrational exuberance that currently surrounds the "big data" or nosql systems communities. This area is experiencing a boom—but nothing takes us from boom to bust like unrealistic hype and broken promises. We should be a little more honest about where these systems already shine and where they are still maturing.

THE THREE TECTONIC PLATES OF SELF-HELP

> **"As to methods there may be a million and then some, but principles are few. The man who grasps principles can successfully select his own methods. The man who tries methods, ignoring principles, is sure to have trouble."**
> — Harrington Emerson

In the last few years, my life has become much more of a joy than a grind, and that's because I've focused on three tectonic plates.

Let's take a close look at each.

1. Intention

Individual or Social?

Americans, in particular, worship at the altar of the rugged individualist. There are clear upsides to this. But steeped in a culture – offline and especially online – that puts the self on a pedestal, we can take self-improvement to be an end unto itself: a better self.

But is it an end unto itself? Does it automatically produce good things? I now have my doubts.

Here's one analogy I've drawn for myself.

Let's pretend that life is the game of soccer. You can work on the mechanics of soccer by yourself. You can always get better at dribbling, shooting, and running drills as a solo practitioner. You can read dozens of books, study tape, and earn a PhD in the physics of ball flight. You can post videos of stunning shots on YouTube and get showered by emojis.

But none of this is actually playing the game of soccer.

You can spend your whole life preparing for, instead of playing, the game of life.

But why would anyone, including yours truly, succumb to this?

Subconsciously, it spares you from the messiest but most rewarding game of all: human interaction. Perhaps people hurt or traumatized you long ago. You might also justify the endless polishing, as I did, with some version of "Once I've perfected myself, then I'll be ready for relationships." But here's the rub: that practice is exactly endless. You can always get better at dribbling and penalty kicks.

Digging further, focusing on improving the self is often in service of trying to control the world, especially if things were unpredictable or unstable when growing up. Banish emotion, live by spreadsheets, and all can be well. All can be *controlled*, or so the illusion goes. But as soon as you're interacting with – let alone depending on – other people, control as a construct goes out the window. And so we consciously or subconsciously avoid the messiness. This is also one of the reasons why a lot of optimizing achiever folks have a hard time in intimate relationships.

So how do I think about "self-help" now, having realized all of the above?

It is refreshingly simple: the goal is to build and improve my relationships. The sooner you get on the real field with real players, the sooner you can get to playing soccer and engaging with life. No more auto-fellating, even with the best of intentions. We've evolved over millions of years to be deeply social creatures, and the more you dodge that IN REAL PHYSICAL LIFE, the more you will suffer. This is why solitary confinement in prisons is often considered cruel and unusual punishment… and yet we do it to ourselves all the time.

There are a few questions that help corral this tectonic plate of *intention*:

- How does any given "self-help" help me in my relationships, and how can I apply it with other people *today* or *this week*?

- How can I take the ship out of the harbor and test it where it counts?

2. Audience

Do you have an audience for your self-development? If so, be careful.

Nary a minute can be spent on social media without bumping into a CAPS-rich "HOW X CHANGED MY LIFE" or a photo carousel of an ayahuasca retreat. If only Costa Rica got a dime for every bikini-clad healer under a waterfall!

Welcome to the theater of performative self-help. I won't belabor this, as we've all seen it, but I suggest reading about the insidious creep of audience capture here, and don't forge ahead in the fame game before reading 11 reasons not to become famous. It's hard to put the genie back in the bottle, so you should know what that genie will do to your life.

But the truth is that most of us aren't extreme examples of this. But even minor tendencies in this direction can do extreme damage over time.

these kinds of vendor comparisons usually are), but the biggest thing missing in the comparison is that *you* don't run DynamoDB, Amazon does. That is a huge, huge difference. Amazon is good at this stuff, and has shown that they can (usually) support massively multi-tenant operations with reasonable SLAs, *in practice*.

I really think there is really only one thing to talk about with respect to reliability: continuous hours of successful production operations. That's it. In some ways the most obvious thing, but not typically what you hear when people talk about these systems. I will believe the system can tolerate (some) failures when I see it tolerate those failures; I believe it can run for a year without downtime when i see it run for a year without downtime. I call this empirical reliability (as opposed to theoretical reliability). And getting good empirical reliability is really, really hard. These systems end up being large hunks of monitoring, tests, and operational procedures with a teeny little distributed system strapped to the back.

You see this showing up in discussions of the CAP theorem all the time. The CAP theorem is a useful thing, but it applies more to system design than system implementations. A design is simple enough that you can maybe prove it provides consistency or tolerates partition failures under some assumptions. This is a useful lens to look at system designs. You can't hope to do this kind of proof with an actual system implementation, the thing you run. The difficulty of building these things means it is really unthinkable that these systems are, in actual reality, either consistent, available, *or* partition tolerant—they certainly all have numerous bugs that will break each of these guarantees. I really like this paper that takes the approach of actually trying to calculate the observed consistency of eventually consistent systems—they seem to do it via simulation rather than measurement, which is unfortunate, but the idea is great.

It isn't that system design is meaningless, it is worth discussing the system design as it does act as a kind of limiting factor on certain aspects of reliability and performance as the implementation matures and improves, but don't take it too seriously as guaranteeing *anything*.

So why isn't this kind of empirical measurement more talked about? I don't know. My pet theory is it has to do with the somewhat rigid and deductive mindset of classical computer science. This is inherited from pure math, and conflicts with the attitude in scientific disciplines. It leads to a preference for starting with axioms, and then proving various properties that follow from these axioms. This world view doesn't embrace the kind of empirical measurements you would expect to justify claims about reality (for another example see this great blog post on programming language productivity claims in programming language research). But that is getting off topic. Suffice it to say, when making predictions about how a system will work in the real world I believe in measurements of reality a lot more than arguments from first-principles.

I think we should insist on a little more rigor and empiricism in this area.

And finally these systems usually want lots of machines. And no matter how good you are, some part of operational difficulty always scales with the number of machines.

Let's discuss some real issues. We had a bug in Kafka recently that lead to the server incorrectly interpreting a corrupt request as a corrupt log, and shutting itself down to avoid appending to a corrupt log. Single machine log corruption is the kind of thing that should happen due to a disk error, and bringing down the corrupt node is the right behavior—it shouldn't happen on all the machines at the same time unless all the disks fail at once. But since this was due to corrupt *requests*, and since we had one client that sent corrupt requests, it was able to sequentially bring down all the servers. Oops. Another example is this Linux bug which causes the system to crash after ~200 days of uptime. Since machines are commonly restarted sequentially this lead to a situation where a large percentage of machines went hard down one after another. Likewise any memory management problems—either leaks or GC problems—tend to happen everywhere at once or not at all. Some companies do public post-mortums for major failures and these are a real wealth of failures in systems that aren't supposed to fail. This paper has an excellent summary of HDFS availability at Yahoo—they note how few of the problems are of the kind that high availability for the namenode would solve. This list could go on, but you get the idea.

I have come around to the view that the real core difficulty of these systems is operations, not architecture or design. Both are important but good operations can often work around the limitations of bad (or incomplete) software, but good software cannot run reliably with bad operations. This is quite different from the view of unbreakable, self-healing, self-operating systems that I see being pitched by the more enthusiastic NoSQL hypesters. Worse yet, you can't easily buy good operations in the same way you can buy good software—you might be able to hire good people (if you can find them) but this is more than just people; it is practices, monitoring systems, configuration management, etc.

These difficulties are one of the core barriers to adoption for distributed data infrastructure. LinkedIn and other companies that have a deep interest in doing creative things with data have taken on the burden of building this kind of expertise in-house—we employ committers on Hadoop and other open source projects on both our engineering and operations team, and have done a lot of from-scratch development in this space where there was gaps. This makes it feasible to take full advantage of an admittedly valuable but immature set of technologies, and let's us build products we couldn't otherwise—but this kind of investment only makes sense at a certain size and scale. It may be too high a cost for small startups or companies outside the web space trying to bootstrap this kind of knowledge inside a more traditional IT organization.

This is why people should be excited about things like Amazon's DynamoDB. When DynamoDB was released, the company DataStax that supports and leads development on Cassandra released a feature comparison checklist. The checklist was unfair in many ways (as

Below are a few questions that I've found helpful for nudging this particular tectonic plate in the right direction:

- If you couldn't tell a soul about "the work" you're doing, would you still do it? If not, you're not developing yourself; you're curating yourself.
- How has sharing your personal development created tradeoffs?
- If you *had to* take down 20% of your most popular posts, which would you take down and why?
- Are you describing strong catalysts (psychedelics, The Hoffman Process, you name it) instead of doing the post-session integration that makes them truly valuable?
- Have you become more robust or more fragile by offering your inner workings up to public vote?
- Has your social presence made you more or less of the person you want to be? How would the you of three or five years ago feel about your last year of posts? What about the you of 10 years from now?

3. Assumption

What are the fundamental assumptions behind your doing "the work"?

Let's begin with a Buddhist parable that I first heard from the incredible Jack Kornfield.

> *The old Master points to a big boulder and asks a disciple, "See that large rock over there?"*
>
> *"Yes", says the disciple.*
>
> *"Do you think it's heavy?" continues the Master.*
>
> *"Yes, it's very heavy!" replies the student.*
>
> *"Only if you pick it up," smiles the Master.*

Once again, the fundamental assumption behind self-help is often this: Something is not OK. Something is wrong. Something is not enough. Something needs fixing. If I can't find it, I'll create it.

We've established this. But there is a follow-on assumption that matters a lot.

If I fix the things that aren't OK, all will be well. If I improve myself enough, if I only work hard enough, I can finally eliminate my suffering.

I hate to inform you, but this doesn't work. I'm also thrilled to inform you that this doesn't work. You can stop picking up a lot of boulders.

There is one book that most opened my eyes to this reframe – *Already Free: Buddhism Meets Psychotherapy on the Path of Liberation* by Bruce Tift. It offers a terrifying but ultimately liberating realization: there is no perfect escape from suffering. It doesn't exist. But there is a way to find your long-sought unclenching, and it lies in cultivating your skill of acceptance as much as that of improvement.

Now, I can hear the chorus: *Has Tim gone soft? Given up the good fight? Is he telling everyone to chill after he himself red-lined and got the spoils? How convenient! And…*

Hold on a second. I'm telling you – intelligent acceptance is high-leverage. It's probably one of the highest forms of leverage. This is an approach that helps preserve your energy for where it really matters. My early forays into Stoicism and Seneca The Younger helped set the conditions for my biggest wins from 2004-2010. Still, I only learned a small fraction of what I needed.

So how do you cultivate your skill of acceptance without becoming complacent?

This is a big question and what I love about Bruce's book. Compared to a strictly Western or purely Eastern book, he blends them and offers a surgical guide to using *both* action and acceptance. You don't have to be a bull in a china shop or a cow in the rain; there is a middle path. That middle path is where all the gold is buried.

If the only tool you have is "self-improvement," you'll become a hammer looking for nails in a world that is 50% screws. I tried it. It can create the veneer of success, but it will leave your inner world in turmoil.

Suffice to say, the dual dance is the most joyful. Upgrade your toolkit with that in mind. Read Bruce's book. If it doesn't click, try *Radical Acceptance: Embracing Your Life with the Heart of a Buddha* by Tara Brach, which had a large impact on my life a decade before I found Bruce's book. In a sense, the writing of Seneca prepared me for Tara, which then prepared me for Bruce. So grab them all and thank me later.

If you want serenity, you need to be able to put the Serenity Prayer into practice. Seriously, I read it all the time.

P^N is an upper bound on reliability but one that you could never, never approach in practice. For example Google has a fantastic paper that gives empirical numbers on system failures in Bigtable and GFS and reports empirical data on groups of failures that show rates several orders of magnitude higher than the independence assumption would predict. This is what one of the best system and operations teams in the world can get: your numbers may be far worse.

The actual reliability of your system depends largely on how bug free it is, how good you are at monitoring it, and how well you have protected against the myriad issues and problems it has. This isn't any different from traditional systems, except that the new software is far less mature. I don't mean this disparagingly, I work in this area, it is just a fact. Maturity comes with time and usage and effort. This software hasn't been around for as long as MySQL or Oracle, and worse, the expertise to run it reliably is much less common. MySQL and Oracle administrators are plentiful, but folks experience with, say, serious production Zookeeper operations knowledge are much more rare.

Kernel filesystem engineers say it takes about a decade for a new filesystem to go from concept to maturity. I am not sure these systems will be mature much faster—they are not easier systems to build and the fundamental design space is much less well explored. This doesn't mean they won't be *useful* sooner, especially in domains where they solve a pressing need and are approached with an appropriate amount of caution, but they are not yet by any means a mature technology.

Part of the difficulty is that distributed system software is actually quite complicated in comparison to single-server code. Code that deals with failure cases and is "cluster aware" is extremely tricky to get right. The root of the problem is that dealing with failures effectively explodes the possible state space that needs testing and validation. For example it doesn't even make sense to expect a single-node database to be fast if its disk system suddenly gets really slow (how could it), but a distributed system does need to carry on in the presence of single degraded machine because it has some many machines, one is sure to be degraded. These kind of "semi-failures" are common and very hard to deal with. Correctly testing these kinds of issues in a realistic setting is brutally hard and the newer generation of software doesn't have anything like the QA processes its more mature predecessors had. (If you get a chance get someone who has worked at Oracle to describe to you what kind of testing they do to a line of code that goes into their database before it gets shipped to customers). As a result there are a lot of bugs. And of course these bugs are on all the machines, so they absolutely happen together.

Likewise distributed systems typically require more configuration and more complex configuration because they need to be cluster aware, deal with timeouts, etc. This configuration is, of course, shared; and this creates yet another opportunity to bring everything to its knees.

is a very different thing from traditional infrastructure. All of these properties are critical to large web companies, and are what drove the adoption of horizontally scalable systems like Hadoop, Cassandra, Voldemort, etc. I was the original author of Voldemort and have worked on distributed infrastructure for the last four years or so. So in-so-far as there is a "big data" debate, I am firmly in the "pro-" camp. But one thing you often hear is that this kind of software is more reliable than the traditional alternatives it replaces, and this just isn't true. It is time people talked honestly about this.

You hear this assumption of reliability everywhere. Now that scalable data infrastructure has a marketing presence, it has really gotten bad. Hadoop or Cassandra or what-have-you can tolerate machine failures then they must be unbreakable right? Wrong.

Where does this come from? Distributed systems need to partition data or state up over lots of machines to scale. Adding machines increases the probability that some machine will fail, and to address this these systems typically have some kind of replicas or other redundancy to tolerate failures. The core argument that gets used for these systems is that if a single machine has probability P of failure, and if the software can replicate data N times to survive N-1 failures, and if the machines fail *independently*, then the probability of losing a particular piece of data must be P^N. So for any desired reliability R and any single-node failure probability P you can pick some replication N so that $P^N < R$. This argument is the core motivation behind most variations on replication and fault tolerance in distributed systems. It is true that without this property the system would be hopelessly unreliable as it grew. But this leads people to believe that distributed software is somehow innately reliable, which unfortunately is utter hogwash.

Where is the flaw in the reasoning? Is it the dreaded Hadoop single points of failure? No, it is far more fundamental than that: the problem is the assumption that failures are independent. Surely no belief could possibly be more counter to our own experience or just common sense than believing that there is no correlation between failures of machines in a cluster. You take a bunch of pieces of identical hardware, run them on the same network gear and power systems, have the same people run and manage and configure them, and run the same (buggy) software on all of them. It would be incredibly unlikely that the failures on these machines would be independent of one another in the probabilistic sense that motivates a lot of distributed infrastructure. If you see a bug on one machine, the same bug is on all the machines. When you push bad config, it is usually game over no matter how many machines you push it to.

MASLOW'S HAMBURGER OF NEEDS?

> **"The more one forgets himself—by giving himself to a cause to serve or another person to love—the more human he is and the more he actualizes himself. What is called 'self-actualization' is not an attainable aim at all, for the simple reason that the more one would strive for it, the more he would miss it. In other words, self-actualization is possible only as a side-effect of self-transcendence."**
> — Viktor E. Frankl

How can we easily keep ourselves on the right track?

As I remind myself these days: **It's the relationships, stupid.**

For a nice simple visual, let's revise Maslow's pyramid with all of this in mind. This is easy, as Maslow never drew his model as a rigid pyramid!

He described "classes" of needs that were unfixed, overlapping, and that could reverse in order. And believe it or not, self-actualization was only ever for the "self-actualizing minority." In the 1960s, his work was co-opted by consultants and corporate trainers who needed a progression to sell. True story.

Given all this, and after decades of trial and error, here's where I've landed:

Maslow's Hamburger of Needs.

Ahhh… what? Not to worry. It's the same good ol' Maslow ingredients, but I think of it as a hamburger:

For our purposes, the meat, the whole point of the hamburger, is that middle layer: relationships. That is the center of life. The heartbeat.

As luck would have it, when you improve the heartbeat, it also feeds everything else.

You'll notice that the meat contains Abe's most-important addendum—the sixth level of self-transcendence. Focusing on things bigger than yourself is a critical piece of the ultimate puzzle. Faith, nature, family, meditation, causes that outlive you, etc. – take your pick. But be careful. If you do it to inflate the ego or impress others, it's self-obsession again, not self-transcendence. If you need credit, it doesn't count.

Of course, it should go without saying but the top and bottom layers matter a lot. A hamburger is a giant mess without the bun. Friends will get sick of you crashing on their couch and eating their food.

But the bread and dressing layers exist to serve the middle. That's the payload. Everything is in service of the payload. And the payload circulates benefits back to the edges, and then the cycle repeats. Even if you think this is oversimplified claptrap, temporarily assuming it's true will help you.

What if nearly everything you focused on — calendar, habits, goals — aimed to improve your relational life somehow? What if you took this as a challenge for even a week? Your lens on the world changes dramatically.

You say yes differently.
You say no more clearly.
Your to-do list for life slowly transforms.

What if all that you focused on, all that you do, had to improve that middle layer in some fashion?

It's a damn hard question if you've been on the *self*-help train for a while. I get it.

So let's try something easier: **What if it only changed how you approach your to-do list? Try hamburger-first each day for 1-2 weeks and tell me what happens. Add and do the things that improve your relational life FIRST. Nothing on the list? Create something. It could be as simple as cooking dinner for your spouse, complimenting at least three people a day for a week, or introducing yourself to the barista you see every morning. Getting started is how you get grooving.**

ARE YOU DOING SELF-HELP, OR IS SELF-HELP DOING YOU?

> **For friendship makes prosperity more shining and lessens adversity by dividing and sharing it.**
> — Marcus Tullius Cicero

In his *Moral Letters to Lucilius*, Seneca the Younger famously wrote that "These individuals [who put money at the center of life] have riches just as we say that we 'have a fever,' when really the fever has us."

What if self-help is similar?

Obsessing over the self never provides peace. It cannot make you whole, as you aren't the whole. Becoming whole starts by putting down the rock you didn't even know you were carrying.

Because at the end of the day—and at the end of a Montana night—the point was never yourself.

It was never the pyramid.

It was never the optimization.

It was the people around the fire.

Share this:

- Share on X (Opens in new window) X
- Share on Facebook (Opens in new window) Facebook
- Share on LinkedIn (Opens in new window) LinkedIn
- Email a link to a friend (Opens in new window) Email
- More

Getting Real About Distributed System Reliability

BOREDANDROID · 01 MAR 2026 · SOURCE

There is a lot of hype around distributed data systems, some of it justified. It's true that the internet has centralized a lot of computation onto services like Google, Facebook, Twitter, LinkedIn (my own employer), and other large web sites. It's true that this centralization puts a lot of pressure on system scalability for these companies. Its true that incremental and horizontal scalability is a *deep* feature that requires redesign from the ground up and can't be added incrementally to existing products. It's true that, if properly designed, these systems can be run with no *planned* downtime or maintenance intervals in a way that traditional storage systems make harder. It's also true that software that is explicitly designed to deal with machine failures